EuroSTAR ’97, 24-28 November 1997, Edinburgh UK.

Testing GUI Applications

Paul zGerrard
Systeme Evolutif Limited
9 Cavendish Place
London W1M 0QD
Tel: +44 (0)20 7636 6060
Fax: +44 (0)20 7636 6072

paulg@evolutif.co.uk
http://www.evolutif.co.uk

Abstract

Most clients in client/server systems deliver system functionality using a graphical user interface (GUI). When testing complete systems, the tester must grapple with the additional functionality provided by the GUI. GUIs make testing systems more difficult for many reasons: the event-driven nature of GUIs, unsolicited events, many ways in/many ways out and the infinite input domain problems make it likely that the programmer has introduced errors because he could not test every path.

Available literature on testing GUIs tends to focus on tools as the solution to the GUI testing problem. With few exceptions, papers on this topic that have been presented at EuroSTAR and STAR (in the US) over the last few years have paid little attention to GUI test design but have concentrated on how automated regression-test suites can be built and maintained. Our intention is to attempt to formulate a GUI test strategy to detect errors and use tools to assist this process. If we achieve this, the task of building regression test suites will be made much easier.

This paper describes the GUI testability ‘problem’ and offers guidelines on testing these sticky objects manually and with automated tools. We present a summary of the types of error that occur most often in GUI applications and propose an approach to designing GUI tests that focus on these errors. The approach consists of a series of test types that are described in turn. These techniques are

assembled into a GUI specific test process that can be mapped onto organisations' existing staged test process.

This is an interim paper based on ongoing research and work on client projects.

Prerequisite Key Words: none

Topic Descriptors: Testing, GUI, Graphical User Interfaces

1 Introduction

1.1 GUIs as universal client

GUIs have become the established alternative to traditional forms-based user interfaces. GUIs are the assumed user interface for virtually all systems development using modern technologies. There are several reasons why GUIs have become so popular:

- GUIs provide the standard look and feel of a client operating system.
- GUIs are so flexible that they can be used in most application areas.
- The GUI provides seamless integration of custom and package applications.
- The user has a choice of using the keyboard or a mouse device.
- The user has a more natural interface to applications: multiple windows can be visible simultaneously, so user understanding is improved.
- The user is in control: screens can be accessed in the sequence the user wants at will.

1.2 GUIs v forms

Lets look at the differences between GUIs and forms based interfaces.

Forms Based Applications

In forms-based applications, the forms are arranged in a hierarchical order. Most often, a top-level menu is displayed which offers a selection of options and when one option is chosen, the selected screen is displayed. Often, one menu calls another lower level menu to provide a further level of selection. In large applications or packages, there might be three levels of menus to be navigated, before the required functionality is presented to the user. A two level menu system containing fifteen options per menu can provide rapid access to over two hundred screens.

With forms displayed on a screen, we can only display and therefore interact with one form at a time. Usually, when a new form is displayed, it fills the screen and functionality on the old form is now unavailable. The one-at-a-time mode is the main characteristic of forms-based systems. In some applications navigation is achieved form-to form, by providing command-driven interfaces. In this way, the user avoids having to navigate using the menu system.

Typically, expert users use such command-driven methods, while occasional users adopt the menu-driven approach. In a sophisticated application, expert users may navigate to virtually any system feature from any other, as long as they know what commands are available, wherever they are. Occasional users are left with the problem of always having to navigate through menus. In large, complex systems, this can be a major headache.

In application forms, the fields on the form have a predefined and unchangeable 'tabbing order'. That is, the user may only access the fields in a certain order, regardless of whether any data has been entered into the form fields. Navigation is normally achieved through use of the tab key to move forwards and the backspace key to go backwards.

GUIs

The most obvious characteristic of GUI applications is the fact that the GUI allows multiple windows to be displayed at the same time. Displayed windows are 'owned' by applications and of course, there may be more than one application active at the same time.

Access to features of the systems is provided via three mechanisms. Menu bars provide almost continuous availability of the various features of the systems; buttons and keyboard shortcuts enable the user to navigate and access the various functions of their application.

Windows provide forms-like functionality with fields in which text or numeric data can be entered. But GUIs introduce additional objects such as radio buttons, scrolling lists, check boxes and other graphics that may be displayed or directly manipulated.

The GUI itself manages the simultaneous presentation of multiple applications and windows. Hidden windows in the same or different applications may be brought forward and used. There are few, if any, constraints on the order in which users access GUI windows so users are free to use the features of the system in the way they prefer, rather than the way the developers architected it.

Fields within windows have a tabbing order, but the user is free to use the mouse to change the focus of the application to any field on screen. There are no constraints on the order in which a user may enter data on a screen. To the user,

there are advantages in being able to access fields directly (perhaps to avoid tabbing through many fields that will not change).

In short, GUIs free the user to access system functionality in their preferred way. They have permanent access to all features and may use the mouse, the keyboard or a combination of both to have a more natural dialogue with the system.

1.3 Some testing difficulties

GUIs have brought considerable benefits to developers. They release the developer from the concerns of interface design – in most environments, GUI design standards impose conventions which make one application look very much like another on the same platform.

However, the sophistication and simplicity of a GUI hides the complexity from the user and where development frameworks are used, the programmers too. When testers are presented with a GUI application to test, the hidden complexities become all too obvious. Consequently, testing GUIs is made considerably more difficult. What are the reasons for this?

Event-driven software

The event-driven nature of GUIs presents the first serious testing difficulty. Because users many click on any pixel on the screen, there are many, many more possible user inputs that can occur. The user has an extremely wide choice of actions. At any point in the application, the users may click on any field or object within a window. They may bring another window in the same application to the front and access that. The window may be owned by another application. The user may choose to access an operating system component directly e.g. a system configuration control panel.

The large number of available options mean that the application code must at all times deal with the next event, whatever it may be. In the more advanced development environments, where sophisticated frameworks are being used, many of these events are handled ‘behind the scenes’. With less advanced toolkits, the programmer must write code to handle these events explicitly. Many errors occur because the programmer cannot anticipate every context in which their event handlers are invoked.

Many events such as button clicks cause the focus of the application to move from one feature to another completely unrelated feature. Not only does the selected feature have to deal with a potentially unknown context, the previous feature may be ‘left hanging’ in a partially completed state. The number of potential paths from feature to feature within the application is so high that the

scope for programmers to make errors is dramatically increased. The ‘infinite paths’ problem also makes it extremely unlikely that they will all be tested.

Unsolicited events

Unsolicited events cause problems for programmers and testers. A trivial example would be when a local printer goes off-line, and the operating system puts up a dialog box inviting the user to feed more paper into the printer. A more complicated situation arises where message-oriented middleware might dispatch a message (an event) to remind the client application to redraw a diagram on screen, or refresh a display of records from a database that has changed.

Unsolicited events may occur at any time, so again, the number of different situations that the code must accommodate is extremely high. Testing of unsolicited events is difficult because of the number of test cases may be high but also special test drivers may be necessary to generate such events within the operating systems.

Object oriented

GUIs map very well to the object-oriented paradigm. The desktop, windows and other graphical elements are usually organised into a hierarchy of objects that deal with GUI events. Every object has its own methods (event handlers) and attributes. Typical attributes define the object’s state and usually include:

- Is the object active or inactive (reacts to mouse clicks to take the focus)?
- Appearance e.g. font type, font size, position on screen, dimensions, visible/invisible, colour.
- Contents e.g. text, on/off, true/false, number and values of entries in a list box.

The number of attributes of objects on screen is large. Even for simple text boxes there may be thirty or more attributes. For the majority of objects, these attributes are static: the appearance attributes are defined once and do not change. However, screen objects that contain data entered by the user must accommodate their changing contents but may also have their own event handlers which may perform validation tasks.

Hidden synchronisation and dependencies

It is common for window objects to have some form of synchronisation implemented. For example, if a check box is set to true, a text box intended to accept a numeric value elsewhere in the window may be made inactive or invisible. If a particular radio button is clicked, a different validation rule might be used for a data field elsewhere on the window.

Synchronisation between objects need not be restricted to objects in the same window. For example, a visible window may present customer details including say, ‘date of last order’. Another open window might be used to log customer orders, so if the user creates and confirms an order for the same customer, should the ‘date of last order’ field on the first window be updated? Most users would suggest it should be so the programmer must use the event handling mechanisms to implement the synchronisation functionality. The problem for the tester is ‘where are these dependencies?’

‘Infinite’ input domain

On any GUI application, the user has complete freedom to click with the mouse-pointing device anywhere on the window that has the focus. Although objects in windows have a default tab order, the user may choose to enter data values by clicking on an object and then entering data. In principle, there may be 5,040 different sequences of entering data into seven data fields (this is seven factorial or 7x6x5x4x3x2x1). For more complex screens the numbers grow dramatically. Do we care as testers, through?

Consider the synchronisation situation above (the selected check box and greyed out numeric field). If the user forgets to check the check box, but then proceeds to enter a value in the numeric field, what should happen if the user then remembers to click on the check box? Should the numeric field be cleared? If not, is the data in numeric field written to the database?

Many ways in, many ways out

An obvious consequence of the event-driven nature of GUIs is that for most situations in the application, . How many ways in are there? Should they all be tested? In the majority of situations in GUI applications, there may be ‘many ways out’ also - the user may use a keyboard shortcut, a button click, a menu option, click on another window etc. How many of these should be tested?

For most options within GUI applications, there are three ways of selecting options or implementing functionality: these are keyboard shortcuts, function keys, and mouse movements (buttons or menus). Given that these three mechanisms are available for many options for most of the time, does this mean we must test these features three times over?

Window management

In a GUI environment, users take the standard features of window management and control for granted. These features include window movement, resizing, maximisation, minimisation and closure. These are usually implemented by standard buttons and keyboard commands available on every window. The programmer has control over which standard window controls are available, but

although the operating system handles the window's behaviour, the programmer must handle the impact on the application.

In some circumstances, closing a window before completing a transaction may leave the application or the database in an inconsistent state. The programmer might avoid such complications by disabling all of the standard window buttons and commands. But he might also have made it impossible for the user to reverse or undo certain actions. From the tester's point of view, which standard window controls need to be tested? Where is the dividing line between testing the application and testing the operating system? Do we need to test navigation paths both forwards and backwards?

2 GUI Test Strategy

2.1 Test Principles Applied to GUIs

Our proposed approach to testing GUIs is guided by several principles, most of which should be familiar. By following these principles we will develop a test process which is generally applicable for testing any GUI application. Note that the proposed test approach does not cover white-box testing of application code in any depth. This approach concentrates on GUI errors and using the GUI to exercise tests so is very-oriented toward black-box testing.

Focus on errors to reduce the scope of tests

We intend to categorise errors into types and design test to detect each type of error in turn. In this way, we can focus the testing and eliminate duplication.

Separation of concerns (divide and conquer)

By focusing on particular types of error and designing test cases to detect those errors, we can break up the complex problem into a number of simpler ones.

Test design techniques where appropriate

Traditional black box test techniques that we would use to test forms based applications are still appropriate.

Layered and staged tests

We will organise the test types into a series of test stages. The principle here is that we bring tests of the lowest level of detail in components up front. We implement integration tests of components and test the integrated application last. In this way, we can build the testing up in trusted layers.

Test automation...wherever possible

Automation most often fails because of over-ambition. By splitting the test process into stages, we can seek and find opportunities to make use of automation where appropriate, rather than trying to use automation everywhere.

2.2 High Level Test Process

An outline test process is presented in Figure 1 - The high-level test process. We can split the process into three overall phases: Test Design, Test Preparation and Test Execution. In this paper, we are going to concentrate on the first stage: Test Design, and then look for opportunities for making effective use of automated tools to execute tests.

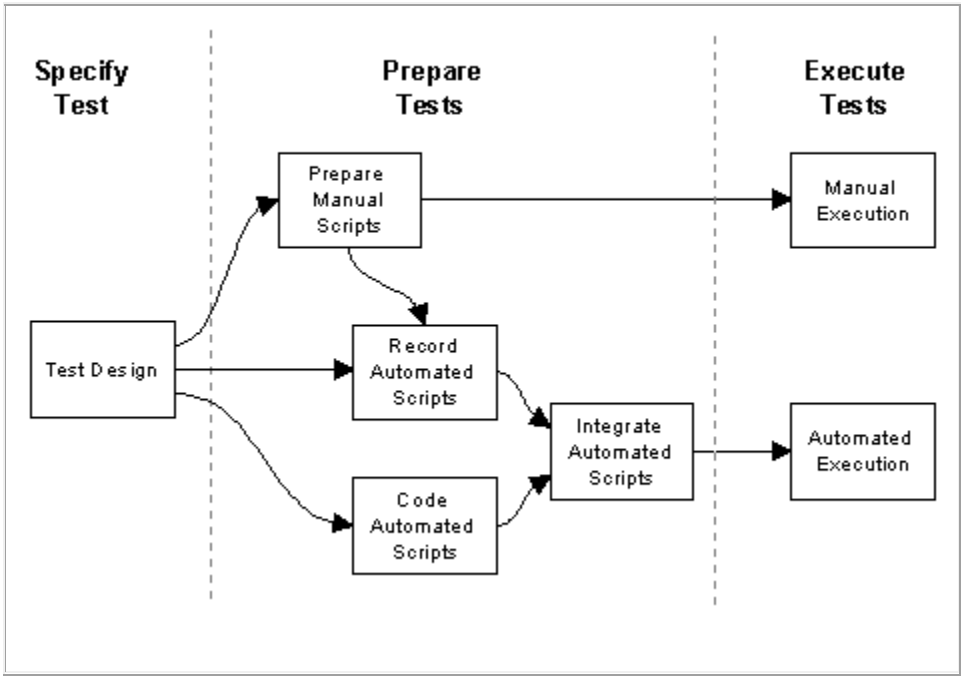

Figure 1 - The high-level test process

2.3 Types of GUI errors

We can list some of the multifarious errors that can occur in a client/server-based application that we might reasonably expect to be able to test for using the GUI. The list in Table 1 is certainly not complete, but it does demonstrate the wide variety error types. Many of these errors relate to the GUI, others relate to the underlying functionality or interfaces between the GUI application and other client/server components.

• Data validation • Incorrect field defaults • Mis-handling of server process failures	• Correct window modality? • Window system commands not available/don't work • Control state alignment with state of

• Mandatory fields, not mandatory • Wrong fields retrieved by queries • Incorrect search criteria • Field order • Multiple database rows returned, single row expected • Currency of data on screens • Window object/DB field correspondence	data in window? • Focus on objects needing it? • Menu options align with state of data or application mode? • Action of menu commands aligns with state of data in window • Synchronisation of window object content • State of controls aligns with state of data in window?

Table 1 - The variety of errors found in GUI applications

By targeting different categories of errors in this list, we can derive a set of different test types that focus on a single error category of errors each and provide coverage across all error types.

2.4 Four Stages of GUI Testing

This paper proposes a GUI test design process that fits into an overall test process. Test design becomes a series of straightforward activities, each focusing on different types of error. The question now is, 'how does this fit into existing test processes?' To help readers map GUI test types to a more traditional test process, we have grouped the test types in four stages.

The four stages are summarised in Table 2 below. We can map the four test stages to traditional test stages as follows:

- Low level - maps to a unit test stage.
- Application - maps to either a unit test or functional system test stage.
- Integration - maps to a functional system test stage.
- Non-functional - maps to non-functional system test stage.

The mappings described above are approximate. Clearly there are occasions when some 'GUI integration testing' can be performed as part of a unit test. The test types in 'GUI application testing' are equally suitable in unit or system testing. In applying the proposed GUI test types, the objective of each test stage, the capabilities of developers and testers, the availability of test environment and tools all need to be taken into consideration before deciding whether and where each GUI test type is implemented in your test process.

The GUI test types alone do not constitute a complete set of tests to be applied to a system. We have not included any code-based or structural testing, nor have we considered the need to conduct other integration tests or non-functional tests of performance, reliability and so on. Your test strategy should address all these issues.

Stage	*Test Types*
Low Level	• Checklist testing • Navigation
Application	• Equivalence Partitioning • Boundary Values • Decision Tables • State Transition Testing
Integration	• Desktop Integration • C/S Communications • Synchronisation
Non-Functional	• Soak testing • Compatibility testing • Platform/environment

Table 2 - Proposed GUI test stages

The test types identified in the table above are described in the following chapter.

3 Types of GUI Test

3.1 Checklist Testing

Checklists are a straightforward way of documenting simple re-usable tests. Not enough use is made of checklist driven testing, perhaps because it is perceived to be trivial, but it is their simplicity that makes them so effective and easy to implement. They are best used to document simple checks that can be made on low-level components. Ideally, these checks can be made visually, or by executing very simple or easily remembered series of commands. The types of checks that are best documented in this way are:

- Programming/GUI standards covering standard features such as:
    - window size, positioning, type (modal/non-modal)
    - standard system commands/buttons (close, minimise, maximise etc.)
- Application standards or conventions such as:
    - standard OK, cancel, continue buttons, appearance, colour, size, location
    - consistent use of buttons or controls
    - object/field labelling to use standard/consistent text.

These checks should be both simple to document and execute and can be used for standalone components so that programmers may make these checks before they release the code for integration.

3.2 Navigation Testing

In the context of a GUI, we can view navigation tests as a form of integration testing. Typically, programmers create and test new windows in isolation. Integration of a new window into an application requires that the application menu definition and invocations of the window from other windows be correctly implemented. The build strategy determines what navigation testing can be done and how. To conduct meaningful navigation tests the following are required to be in place:

- An application backbone with at least the required menu options and call mechanisms to call the window under test.
- Windows that can invoke the window under test.
- Windows that are called by the window under test.

Obviously, if any of the above components are not available, stubs and/or drivers will be necessary to implement navigation tests. If we assume all required components are available, what tests should we implement? We can split the task into steps:

- For every window, identify all the legitimate calls to the window that the application should allow and create test cases for each call.
- Identify all the legitimate calls from the window to other features that the application should allow and create test cases for each call.
- Identify reversible calls, i.e. where closing a called window should return to the ‘calling’ window and create a test case for each.
- Identify irreversible calls i.e. where the calling window closes before the called window appears.

There may be multiple ways of executing a call to another window i.e. menus, buttons, keyboard commands. In this circumstance, consider creating one test case for each valid path by each available means of navigation.

Note that navigation tests reflect only a part of the full integration testing that should be undertaken. These tests constitute the ‘visible’ integration testing of the GUI components that a ‘black box’ tester should undertake.

3.3 Application Testing

Application testing is the testing that would normally be undertaken on a forms-based application. This testing focuses very much on the behaviour of the objects within windows. The approach to testing a window is virtually the same

as would be adopted when testing a single form. The traditional black-box test design techniques are directly applicable in this context.

Given the extensive descriptions already available (BEIZER, KANER, MYERS) no explanation of these techniques is provided here. However, a very brief summary of the most common techniques and some guidelines for their use with GUI windows are presented in the table below:

Technique	***Used to test***
Equivalence Partitions and Boundary Value Analysis	• Input validation • Simple rule-based processing
Decision Tables	• Complex logic or rule-based processing
State-transition testing	• Applications with modes or states where processing behaviour is affected • Windows where there are dependencies between objects in the window.

Table 3 - Traditional test techniques

3.4 Desktop Integration Testing

It is rare for a desktop PC or workstation to run a single application. Usually, the same machine must run other bespoke applications or shrink wrapped products such as a word processor, spreadsheet, electronic mail or Internet based applications. Client/server systems assume a ‘component based’ architecture so they often treat other products on the desktop as components and make use of features of these products by calling them as components directly or through specialist middleware.

We define desktop integration as the integration and testing of a client application with these other components. Because these interfaces may be hidden or appear ‘seamless’ when working, the tester usually needs to understand a little more about the technical implementation of the interface before tests can be specified. The tester needs to know what interfaces exist, what mechanisms are used by these interfaces and how the interface can be exercised by using the application user interface.

To derive a list of test cases the tester needs to ask a series of questions for each known interface:

- Is there a dialogue between the application and interfacing product (i.e. a sequence of stages with different message types to test individually) or is it a direct call made once only?

- Is information passed in both directions across the interface?
- Is the call to the interfacing product context sensitive?
- Are there different message types? If so, how can these be varied?

In principle, the tester should prepare test cases to exercise each message type in circumstances where data is passed in both directions. Typically, once the nature of the interface is known, equivalence partitioning, boundary values analysis and other techniques can be used to expand the list of test cases.

3.5 Client/Server Communication Testing

Client/Server communication testing complements the desktop integration testing. This aspect covers the integration of a desktop application with the server-based processes it must communicate with. The discussion of the types of test cases for this testing is similar to section 3.4 Desktop Integration, except there should be some attention paid to testing for failure of server-based processes.

In the most common situation, clients communicate directly with database servers. Here the particular tests to be applied should cover the various types of responses a database server can make. For example:

- Logging into the network, servers and server-based DBMS.
- Single and multiple responses to queries.
- Correct handling of errors (where the SQL syntax is incorrect, or the database server or network has failed)
- Null and high volume responses (where no rows or a large number of rows are returned).

The response times of transactions that involve client/server communication may be of interest. These tests might be automated, or timed using a stopwatch, to obtain indicative measures of speed.

3.6 Synchronisation Testing

There may be circumstances in the application under test where there are dependencies between different features. One scenario is when two windows are displayed, a change is made to a piece of data on one window and the other window needs to change to reflect the altered state of data in the database. To accommodate such dependencies, there is a need for the dependent parts of the application to be synchronised.

Examples of synchronisation are when:

- The application has different modes - if a particular window is open, then certain menu options become available (or unavailable).

- If the data in the database changes and these changes are notified to the application by an unsolicited event to update displayed windows.
- If data on a visible window is changed and makes data on another displayed window inconsistent.

In some circumstances, there may be reciprocity between windows. For example, changes on window A trigger changes in window B and the reverse effect also applies (changes in window B trigger changes on window A).

In the case of displayed data, there may be other windows that display the same or similar data which either cannot be displayed simultaneously, or should not change for some reason. These situations should be considered also. To derive synchronisation test cases:

- Prepare one test case for every window object affected by a change or unsolicited event and one test case for reciprocal situations.
- Prepare one test case for every window object that must not be affected - but might be.

3.7 Non-Functional Testing

The tests described in the previous sections are functional tests. These tests are adequate for demonstrating the software meets it's requirements and does not fail. However, GUI applications have non-functional modes of failure also. We propose three additional GUI test types (that are likely to be automated).

Soak Testing

In production, systems might be operated continuously for many hours. Applications may be comprehensively tested over a period of weeks or months but are not usually operated for extended periods in this way. It is common for client application code and bespoke middleware to have memory-leaks. Soak tests exercise system transactions continuously for an extended period in order to flush out such problems.

These tests are normally conducted using an automated tool. Selected transactions are repeatedly executed and machine resources on the client (or the server) monitored to identify resources that are being allocated but not returned by the application code.

Compatibility Testing

Whether applications interface directly with other desktop products or simply co-exist on the same desktop, they share the same resources on the client. Compatibility Tests are (usually) automated tests that aim to demonstrate that

resources that are shared with other desktop products are not locked unnecessarily causing the system under test or the other products to fail.

These tests normally execute a selected set of transactions in the system under test and then switch to exercising other desktop products in turn and doing this repeatedly over an extended period.

Platform/Environment Testing

In some environments, the platform upon which the developed GUI application is deployed may not be under the control of the developers. PC end-users may have a variety of hardware types such as 486 and Pentium machines, various video drivers, Microsoft Windows 3.1, 95 and NT. Most users have PCs at home nowadays and know how to customise their PC configuration. Although your application may be designed to operate on a variety of platforms, you may have to execute tests of these various configurations to ensure when the software is implemented, it continues to function as designed. In this circumstance, the testing requirement is for a repeatable regression test to be executed on a variety of platforms and configurations. Again, the requirement for automated support is clear so we would normally use a tool to execute these tests on each of the platforms and configurations as required.

4 Test Automation

4.1 Justifying Automation

Automating test execution is normally justified based on the need to conduct functional regression tests. In organisations currently performing regression test manually, this case is easy to make - the tool will save testers time. However, most organisations do not conduct formal regression tests, and often compensate for this ‘sub-consciously’ by starting to test late in the project or by executing tests in which there is a large amount of duplication.

In this situation, buying a tool to perform regression tests will not save time, because no time is being spent on regression testing in the first place. In organisations where development follows a RAD approach or where development is chaotic, regression testing is difficult to implement at all - software products may never be stable enough for a regression test to mature and be of value. Usually, the cost of developing and maintaining automated tests exceeds the value of finding regression errors.

We propose that by adopting a systematic approach to testing GUIs and using tools selectively for specific types of tests, tools can be used to find errors during the early test stages. That is, we can use tools to find errors pro-actively rather than repeating tests that didn’t find bugs first time round to search for regression errors late in a project.

4.2 Automating GUI Tests

Throughout the discussion of the various test types in the previous chapter, we have assumed that by designing tests with specific goals in mind, we will be in a better position to make successful choices on whether we automate tests or continue to execute them manually. Based on our experience of preparing automated tests and helping client organisations to implement GUI test running tools we offer some general recommendations concerning GUI test automation below.

Pareto law	• We expect 80% of the benefit to derive from the automation of 20% of the tests. • Don't waste time scripting low volume complex scripts at the expense of high volume simple ones.
Hybrid Approach	• Consider using the tools to perform navigation and data entry prior to manual test execution. • Consider using the tool for test running, but perform comparisons manually or 'off-line'.
Coded scripts	• These work best for navigation and checklist-type scripts. • Use where loops and case statements in code leverage simple scripts. • Are relatively easy to maintain as regression tests.
Recorded Scripts	• Need to be customised to make repeatable. • Sensitive to changes in the user interface.
Test Integration	• Automated scripts need to be integrated into some form of test harness. • Proprietary test harnesses are usually crude so custom-built harnesses are required.
Migrating Manual Test Scripts	• Manual scripts document automated scripts • Delay migration of manual scripts until the software is stable, and then reuse for regression tests.
Non-Functional Tests	• Any script can be reused for soak tests, but they must exercise the functionality of concern. • Tests of interfaces to desktop products and server processes are high on the list of tests to automate. • Instrument these scripts to take response time measurements and re-use for performance testing.

From the discussion above, we are now in a position to propose a test automation regime that fits the GUI test process. Table 4 - Manual versus automated execution presents a rough guideline and provides a broad indication of our recommended approach to selecting tests to automate.

Test Types	***Manual or Automated?***
Checklist testing	Manual execution of tests of application conventions Automated execution of tests of object states, menus and standard features
Navigation	Automated execution.
Equivalence Partitioning, Boundary Values, Decision Tables, State Transition Testing	Automated execution of large numbers of simple tests of the same functionality or process e.g. the 256 combinations indicated by a decision table. Manual execution of low volume or complex tests
Desktop Integration, C/S Communications	Automated execution of repeated tests of simple transactions Manual tests of complex interactions
Synchronisation	Manual execution.
Soak testing, Compatibility testing, Platform/environment	Automated execution.

Table 4 - Manual versus automated execution

5 Improving the testability of GUI Applications

5.1 The GUI Testing Challenge

It is clear that GUIs present a challenge to testers because they appear to be inherently more difficult to test. The flexibility of GUIs invites programmers to pass on this flexibility to end users in their applications. Consequently, users can exercise the application code in ways never envisaged by the programmers and which are likely to be released untested.

If testability is the ease with which a tester can specify, prepare, execute and analyse tests, it is arguable that it is possible for programmers to build untestable systems using GUIs.

It is difficult to specify tests because much of the underlying functionality in a GUI application is undocumented. Because of the event-driven nature of GUIs, a considerable amount of programming effort is expended on dealing with hidden interactions that come to light during the informal programmer testing so tend to go undocumented.

It is difficult to prepare tests because the number tests required to exercise paths through the application which a user might follow has escalated dramatically. If we consider using menus, function keys and mouse movements to exercise system features, the number of tests increased further.

It is difficult to execute tests*.* Using a manual pointing device is virtually unrepeatable and certainly error prone. Creating tests which stimulate hidden interactions, set or amend visible (or invisible) GUI objects is troublesome. Separating tests of application code from the GUI elements of the operating system is tricky.

It is difficult to analyse tests because there is constant change on the screen and behind the screen. Windows on which results are displayed may appear and all other visible windows may be refreshed simultaneously making visual inspection difficult. Expected results may not be directly displayed but on hidden windows. Attributes of objects to be verified may be invisible or difficult to detect by eye. Windows that display invalid results may be hidden by other windows or on windows that are minimised.

5.2 GUI Design for Testability

We make the following recommendations to GUI designers aimed at improving testability. We suggest that the most straightforward way of implementing them is to include checks on these design issues in checklist test cases. Some of these recommendations impact the freedom users have to use software in certain ways, but we believe that if the application structure and organisation is well designed, the user will have little need to make unusual choices.

1. Where applications have modes of operation so that some features become meaningless or redundant, then these options on menus should be greyed-out or disabled in some other way.
2. Unless there are specific requirements to display the same data on multiple windows the designer should avoid having to build in dependencies between windows to eliminate ‘displayed data’ inconsistencies.
3. Navigation between windows should be hierarchic, (in preference to anarchic) to minimise the number of windows that might be open at once and to reduce the number of paths through the system.

4. Unless there is an impact on usability, windows should be modal to reduce the number of paths through the system and reduce window testing to a simpler, forms-like test process.
5. Unless there is an impact on usability, dependencies between objects on windows should be avoided or circumvented by splitting user transactions into multiple modal windows.
6. The number of system commands (maximise, minimise, close, restore) available on windows should be reduced to a minimum.
7. Functionality which is accessed by equivalent button clicks, function keys and menu options should be implemented using the same function-call to reduce the possibility of errors and the need to always test all three mechanisms.
8. Instrumentation should be implemented in code to provides information on application interfaces to other desktop products or server-based processes and should be an option which can be turned on or off by testers.
9. Instrumentation should be implemented to provide information on the content of unsolicited events from other applications and also to simulate these unsolicited events for test purposes.

6 Conclusion

In summary, GUIs have introduced new types of error, increased complexity and made testing more difficult. Systematic test design helps us to focus on the important tests and gives us an objective way of addressing risks. Tools are appropriate for many but not all tests and a staged approach to testing enables us to identify which tests to automate much more easily. Tools can therefore be used to detect errors pro-actively as well as to execute regression tests.

This is an interim paper based on ongoing research and work on client projects. There is more work to be done to refine the approach to turn it into a portable and flexible GUI test strategy. Our ambition is to create a GUI testing framework that can be part of an integrated client/server or distributed system test strategy that provides distinct opportunities for the implementation of automated testing tools.

7 References

BEIZER	"Software Testing Techniques" by Boris Beizer, Van Nostrand Reinhold, 2nd edition, 1990
KANER	"Testing Computer Software" by Kaner, Falk and Nguyen, Van Nostrand Reinhold, 2nd edition, 1993

MYERS

"The Art of Software Testing" by Glenford J Myers, Wiley, 1979